

WOXSEN UNIVERSITY

School of Technology

PROJECT REPORT

on

Design and Implementation of a Database Management System for IT Helpdesk & Asset Support

Submitted in partial fulfillment of the requirements for the degree of
B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

Dasuri Sohan Yadav [25WU0102269]

ROLL NUMBER	25WU0102269
-------------	-------------

COURSE	Database Management Systems
--------	-----------------------------

FACULTY	Dr. Kiran Mayee
---------	-----------------

ACADEMIC YEAR	2026
---------------	------

Woxsen University, School of Technology
2026

Contents

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary (Table Structure, Data Types, Constraints)
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs (Including the Presentation-II Query)
11. UI Design and Screenshots
12. Implementation Details (Technology Stack, DB Connectivity, Key Code Snippets)
13. Testing (Test Cases and Results)
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2 Abstract

IT helpdesks handle a constant stream of incidents, service requests and hardware/software assets. When this information lives in spreadsheets and e-mail threads, tickets lose their history, assets cannot be audited and service-level (SLA) deadlines are missed unnoticed. This project designs and implements a relational Database Management System for an **IT Helpdesk and Asset Support Management System** that acts as a single source of truth linking every request to the user who raised it, the technician who handles it, the asset involved, its category and the SLA that governs it.

The work proceeded in three stages. First, the requirements and an Entity–Relationship model with six entities - **User, Ticket, Asset, Technician, Category** and **SLA Policy** - were produced. Second, the ER model was mapped to a relational schema in Third Normal Form and implemented in MySQL with primary keys, foreign keys, CHECK, UNIQUE and NOT NULL constraints, then loaded with sample data and exercised through multi-table JOIN, aggregate and subquery queries. Third, a Flask web application with an HTML/CSS/JavaScript front end was connected to the database to provide role-based Insert, View and Delete operations for Admins, Technicians and Users.

Testing shows that the constraints reject invalid data (orphan foreign keys, duplicate keys, out-of-domain values), that deletions propagate safely through `ON DELETE` rules, and that every change made in the interface is immediately visible in the database.

Keywords: DBMS, ER model, normalization, 3NF, MySQL, SQL joins, IT helpdesk, asset management, Flask, referential integrity.

3 Introduction and Problem Statement

3.1 Background

Every organisation depends on laptops, monitors, printers, software licences and the support staff who keep them running. An IT helpdesk receives requests such as **“my laptop will not boot”** or **“the VPN keeps disconnecting”**, routes them to a technician, and must resolve them within an agreed time. Doing this well requires data that is connected: who reported the problem, which device is affected, what its warranty status is, who is working on it and whether the deadline will be met.

3.2 Problem statement

Many small and mid-sized helpdesks still track work in spreadsheets, shared mailboxes and chat. This leads to:

- **Redundant and inconsistent data** - the same user, device or technician typed repeatedly with variations.
- **No link between a ticket and an asset** - warranty and history must be hunted down manually.
- **Unreliable SLA tracking** - no structured priority-to-deadline rule, so breaches go unseen. **Poor**
- **access control and auditability** - anyone with the file can see or alter everything.
- **Weak reporting** - workload, category trends and asset audits require manual counting.

The problem addressed here is to **design and implement a normalized relational database, and a working interface on top of it, that integrates tickets, assets, users, technicians, categories and SLA rules while enforcing data integrity through keys and constraints.**

3.3 Real-world walk-through

An employee's laptop will not boot and she submits a request through the portal. (1) A new row is created in `tickets` with her `user_id`, category **Hardware** and priority **High**. (2) `tickets.asset_id` joins to `assets`, giving model, serial number and warranty. (3) A technician is assigned through `technician_id`. (4) The `sla_policy` row for **High** priority sets a 4-hour resolution target. (5) When the ticket is closed the outcome remains in the database as a reportable record. The result is one linked record set - no re-entry, no lost history.

4 Objectives and Scope

4.1 Objectives

1. **Streamline ticketing** - log, assign and track tickets from creation to closure.
2. **Track assets** - keep a lifecycle record (purchase, warranty, status) for every hardware asset.
3. **Enforce data integrity** - use primary/foreign keys, constraints and normalization to remove redundancy.
4. **Support SLA monitoring** - store response and resolution targets per priority so deadlines can be computed by query.
5. **Provide role-based access** - separate Admin, Technician and User views of the same data.
6. **Deliver a working interface** - a web UI performing Insert, View and Delete against the live database.

4.2 Functional modules

Module	Purpose
Ticket Management	Create, categorize, prioritize and route tickets to the right technician.
Asset Registry	Track hardware assets, warranties and status through their lifecycle.
User & Role Access	Manage end users, technicians and admins with role-based permissions.
SLA & Escalation	Hold response/resolution targets per priority to detect breaches.
Reporting & Audit	Queries for workload, category trends, warranty expiry and SLA deadlines.
Knowledge Base (future)	Link recurring issues to documented solutions.

4.3 Scope

In scope

- Six-table relational schema in 3NF on MySQL
- Sample data, DDL, DML and analytical queries
- Flask web UI with login, Insert, View, Delete
- Hashed passwords, role-based routes, delete confirmation

Out of scope (this version)

- E-mail/SMS notifications and automatic escalation jobs
- Software-licence and consumables tracking
- Mobile app and external API integration
- Knowledge-base module and AI-based triage

5 Software and Hardware Requirements

5.1 Software requirements

Component	Specification
Database server	MySQL 8.0 (MySQL-compatible syntax; PostgreSQL also viable)
Database client	MySQL Command-Line Client / MySQL Workbench
Backend language / framework	Python 3, Flask
DB connectivity	mysql-connector-python
Security library	Werkzeug (bcrypt password hashing)
Front end	HTML5, CSS3, JavaScript (Jinja2 templates)
Operating system	Windows 10/11, Ubuntu 20.04+ or macOS
Browser	Any current Chrome, Edge or Firefox
Version control	Git and GitHub

5.2 Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 2 GHz	Quad-core 2.5 GHz or better
Memory (RAM)	4 GB	8 GB or more
Storage	2 GB free	10 GB free (SSD)
Display	1366 × 768	1920 × 1080
Network	Localhost / LAN	LAN or cloud hosting (AWS / Azure) for deployment

6ER Diagram

The ER model has six entities. **Ticket** is the central entity; each of the other five connects to it through a one-to-many relationship. The foreign keys therefore all live on the **many** side, in tickets.

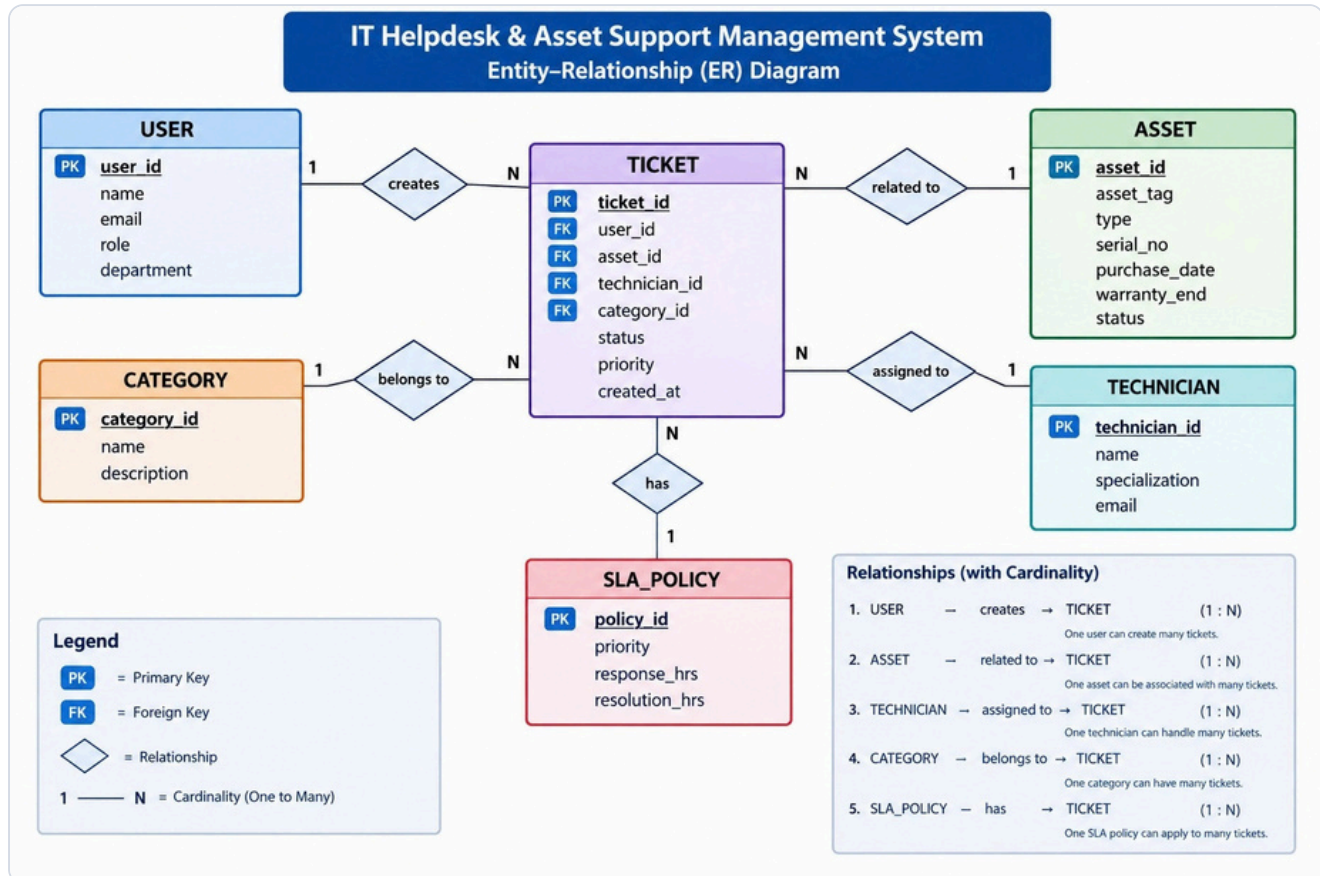


Figure 6.1 — Entity-Relationship diagram of the IT Helpdesk & Asset Support Management System

#	Relationship	Cardinality	Meaning
1	USER — creates → TICKET	1 : N	One user can create many tickets.
2	ASSET — related to → TICKET	1 : N	One asset can be associated with many tickets.
3	TECHNICIAN — assigned to → TICKET	1 : N	One technician can handle many tickets.
4	CATEGORY — belongs to → TICKET	1 : N	One category can have many tickets.
5	SLA_POLICY — has → TICKET	1 : N	One SLA policy (per priority) can apply to many tickets.

Naming. The ER diagram uses singular entity names (USER, TICKET...). In the implemented MySQL database the tables are named in the plural (users, tickets, assets, technicians, categories, sla_policy), matching the working application.

In the implemented MySQL technicians, categories,

7 Relational Schema and Normalization

7.1 Mapping ER to relational schema

Each entity becomes a table with its identifier as primary key. Because every relationship is 1 : N, no separate junction table is needed; the primary key of the "one" side is stored as a foreign key in tickets. (Primary keys underlined, foreign keys in *italics*.)

sla_policy	(policy_id, priority [UNIQUE], response_hrs, resolution_hrs)
categories	(category_id, name [UNIQUE], description)
users	(user_id, name, email [UNIQUE], password_hash, role, department)
technicians	(technician_id, name, specialization, email [UNIQUE])
assets	(asset_id, asset_tag [UNIQUE], type, serial_no [UNIQUE], purchase_date, warranty_end, status)
tickets	(ticket_id, user_id → users, asset_id → assets, technician_id → technicians, category_id → categories, status, priority → sla_policy.priority, created_at)

Referential actions

Foreign key	References	ON DELETE	Rationale
tickets.user_id	users(user_id)	RESTRICT	A user who has raised tickets cannot be deleted, preserving history.
tickets.category_id	categories(category_id)	RESTRICT	A category in use cannot be removed.
tickets.asset_id	assets(asset_id)	SET NULL	A retired/removed asset can be deleted; the ticket history is kept with no asset.
tickets.technician_id	technicians(technician_id)	SET NULL	A departing technician leaves tickets unassigned rather than deleting them.
tickets.priority	sla_policy(priority)	— (ON UPDATE CASCADE)	Priority must be a defined SLA level; renaming a level updates all tickets.

7.2 Normalization

To see why the design is split into six tables, consider the un-normalized record a spreadsheet-based helpdesk would keep, with one wide row per ticket:

```
TICKET_SHEET( ticket_id, user_name, user_email, user_dept, asset_tag, asset_type, serial_no,  
warranty_end, technician_name, technician_email, specialization,  
category_name, priority, response_hrs, resolution_hrs, status, created_at )
```

This design repeats user, asset and technician details on every ticket, which causes update, insertion and deletion anomalies (changing a technician's e-mail means editing many rows; a new asset cannot be recorded until a ticket exists for it). The steps below remove these problems.

Normal form	Rule	How the schema satisfies it
1NF	Atomic values, no repeating groups, a primary key.	Every column holds a single value (e.g. <code>name</code> , <code>email</code> and <code>department</code> are separate columns). Each table has a single-column primary key.
2NF	1NF and no partial dependency of a non-key attribute on part of a composite key.	All primary keys are single-column surrogate keys, so partial dependencies cannot occur. Attributes of users, assets and technicians were moved out of the ticket row into their own tables, each depending on its whole key.
3NF	2NF and no transitive dependency (non-key $\rightarrow$ non-key).	In the wide sheet, <code>priority</code> $\rightarrow$ <code>response_hrs</code> , <code>resolution_hrs</code> and <code>category_id</code> $\rightarrow$ <code>category_name</code> are transitive through <code>ticket_id</code> . They are removed by creating <code>sla_policy</code> and <code>categories</code> . Likewise <code>asset_id</code> $\rightarrow$ <code>asset_tag</code> , <code>type</code> , <code>warranty_end...</code> , <code>user_id</code> $\rightarrow$ <code>name</code> , <code>email...</code> and <code>technician_id</code> $\rightarrow$ <code>name...</code> are stored once in their own tables.

Functional dependencies of the final schema

- `user_id` $\rightarrow$ `name`, `email`, `password_hash`, `role`, `department`
- `technician_id` $\rightarrow$ `name`, `specialization`, `email`
- `asset_id` $\rightarrow$ `asset_tag`, `type`, `serial_no`, `purchase_date`, `warranty_end`, `status`
- `category_id` $\rightarrow$ `name`, `description`
- `policy_id` $\rightarrow$ `priority`, `response_hrs`, `resolution_hrs` and `priority` $\rightarrow$ `policy_id` (candidate key)
- `ticket_id` $\rightarrow$ `user_id`, `asset_id`, `technician_id`, `category_id`, `status`, `priority`, `created_at`

In every dependency the determinant is a candidate key of its table, so the schema is in 3NF (and in practice BCNF).

8 Data Dictionary (Table Structure, Data Types, Constraints)

8.1 users

People who log in to the system: end users, technicians and admins.

Column	Data type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(60)	NOT NULL	Full name
email	VARCHAR(60)	NOT NULL, UNIQUE	Login e-mail
password_hash	VARCHAR(255)	NOT NULL	Werkzeug script hash (no plain text)
role	ENUM('Admin','Technician','User')	NOT NULL, DEFAULT 'User'	Access level
department	VARCHAR(40)	NULL allowed	Department of the user

8.2 technicians

Support staff to whom tickets are assigned.

Column	Data type	Constraints	Description
technician_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique technician identifier
name	VARCHAR(60)	NOT NULL	Technician name
specialization	VARCHAR(40)	NULL allowed	Area of expertise
email	VARCHAR(60)	NOT NULL, UNIQUE	Contact e-mail

8.3 assets

Hardware assets under management.

Column	Data type	Constraints	Description
asset_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique asset identifier
asset_tag	VARCHAR(30)	NOT NULL, UNIQUE	Inventory label, e.g. AST-1001
type	VARCHAR(30)	NOT NULL	Laptop, Monitor, Printer...
serial_no	VARCHAR(40)	NOT NULL, UNIQUE	Manufacturer serial number

Column	Data type	Constraints	Description
purchase_date	DATE	NULL allowed	Date of purchase
warranty_end	DATE	CHECK (warranty_end ≥ purchase_date)	Warranty expiry date
status	ENUM('In Use','In Stock','Under Repair','Retired')	NOT NULL, DEFAULT 'In Stock'	Lifecycle state

8.4 categories

Classification of tickets.

Column	Data type	Constraints	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category identifier
name	VARCHAR(40)	NOT NULL, UNIQUE	Category name
description	VARCHAR(120)	NULL allowed	What the category covers

8.5 sla_policy

Response/resolution targets per priority level.

Column	Data type	Constraints	Description
policy_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique policy identifier
priority	VARCHAR(10)	NOT NULL, UNIQUE	Low / Medium / High / Critical
response_hrs	INT	NOT NULL, CHECK (> 0)	Hours allowed to first respond
resolution_hrs	INT	NOT NULL, CHECK (≥ response_hrs)	Hours allowed to resolve

8.6 tickets

Central table: one row per support request.

Column	Data type	Constraints	Description
ticket_id	INT	PRIMARY KEY, AUTO_INCREMENT (starts 5001)	Unique ticket number
user_id	INT	NOT NULL, FK → users(user_id) ON DELETE RESTRICT	Who raised the ticket
asset_id	INT	NULL, FK → assets(asset_id) ON DELETE SET NULL	Affected asset
technician_id	INT	NULL, FK → technicians(technician_id) ON DELETE SET NULL	Assigned technician (NULL = unassigned)
category_id	INT	NOT NULL, FK → categories(category_id) ON DELETE RESTRICT	Ticket category

Column	Data type	Constraints	Description
status	ENUM('Open','In Progress','Closed')	NOT NULL, DEFAULT 'Open'	Workflow state
priority	VARCHAR(10)	NOT NULL, DEFAULT 'Medium', FK → sla_policy(priority)	Priority / SLA level
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Time the ticket was raised

9SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Data Definition Language (DDL)

The complete schema script (`database.sql`) is shown below. Tables are created parents-first so that foreign keys can be resolved.

```
CREATE DATABASE it_helpdesk;
USE it_helpdesk;

CREATE TABLE sla_policy (
  policy_id      INT AUTO_INCREMENT PRIMARY KEY,
  priority       VARCHAR(10) NOT NULL UNIQUE,
  response_hrs   INT NOT NULL CHECK (response_hrs > 0),
  resolution_hrs INT NOT NULL CHECK (resolution_hrs >= response_hrs)
);

CREATE TABLE categories (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(40) NOT NULL UNIQUE,
  description VARCHAR(120)
);

CREATE TABLE users (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(60) NOT NULL,
  email        VARCHAR(60) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role         ENUM('Admin','Technician','User') NOT NULL DEFAULT 'User',
  department   VARCHAR(40)
);

CREATE TABLE technicians (
  technician_id INT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(60) NOT NULL,
  specialization VARCHAR(40),
  email         VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE assets (
  asset_id      INT AUTO_INCREMENT PRIMARY KEY,
  asset_tag     VARCHAR(30) NOT NULL UNIQUE,
  type         VARCHAR(30) NOT NULL,
  serial_no     VARCHAR(40) NOT NULL UNIQUE,
  purchase_date DATE,
  warranty_end   DATE,
  status        ENUM('In Use','In Stock','Under Repair','Retired') NOT NULL DEFAULT 'In Stock',
  CHECK (warranty_end >= purchase_date)
);

CREATE TABLE tickets (
  ticket_id      INT AUTO_INCREMENT PRIMARY KEY,
  user_id        INT NOT NULL,
  asset_id       INT NOT NULL,
  technician_id  INT NULL,
  category_id    INT NOT NULL,
  status         ENUM('Open','In Progress','Closed') NOT NULL DEFAULT 'Open',
  priority       VARCHAR(10) NOT NULL DEFAULT 'Medium',
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_t_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE RESTRICT,
  CONSTRAINT fk_t_asset FOREIGN KEY (asset_id) REFERENCES assets(asset_id) ON DELETE SET NULL,
  CONSTRAINT fk_t_tech FOREIGN KEY (technician_id) REFERENCES technicians(technician_id) ON DELETE SET NULL,
  CONSTRAINT fk_t_cat FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE RESTRICT,
  CONSTRAINT fk_t_sla FOREIGN KEY (priority) REFERENCES sla_policy(priority) ON UPDATE CASCADE
);
```

```

) AUTO_INCREMENT = 5001;

CREATE INDEX idx_tickets_status_priority ON tickets(status, priority);

```

9.2 Data Manipulation Language (DML) - loading sample data

Sample rows were loaded with `INSERT` statements and confirmed with `SELECT *`. (Rows for assets 101–103 and tickets 5001–5003 are those used in Presentation II; the remaining rows are illustrative.)

```

INSERT INTO sla_policy (priority, response_hrs, resolution_hrs) VALUES
('Low',24,72), ('Medium',8,24), ('High',1,4), ('Critical',1,2);

INSERT INTO categories (name, description) VALUES
('Hardware','Laptop, desktop and boot or power faults'),
('Display & Peripherals','Monitors, printers, keyboards and mice'),
('Software','Installation, licensing and application errors'),
('Network','Wi-Fi, VPN and connectivity problems');

INSERT INTO users (user_id, name, email, password_hash, role, department) VALUES
(1,'System Admin','admin@company.com','<scrypt hash>','Admin','IT'),
(3,'Marcus Lee','marcus.lee@company.com','<scrypt hash>','Technician','IT'),      (4,'Priya
Nair','priya.nair@company.com','<scrypt hash>','Technician','IT'),      (12,'Anita
Rao','anita.rao@company.com','<scrypt hash>','User','Finance'),      (15,'Rahul
Verma','rahul.verma@company.com','<scrypt hash>','User','Marketing'),      (18,'Sneha
Iyer','sneha.iyer@company.com','<scrypt hash>','User','HR');

INSERT INTO technicians (technician_id, name, specialization, email) VALUES
(3,'Marcus Lee','Hardware','marcus.lee@company.com'),
(4,'Priya Nair','Software & Peripherals','priya.nair@company.com');

INSERT INTO assets VALUES
(101,'AST-1001','Laptop','SN-88213X','2024-03-15','2027-03-15','In Use'),
(102,'AST-1002','Monitor','SN-77120Y','2023-11-02','2026-11-02','In Use'),
(103,'AST-1003','Printer','SN-55391Z','2021-06-30','2025-06-30','Retired');

INSERT INTO tickets (ticket_id,user_id,asset_id,technician_id,category_id,status,priority,created_at)
VALUES
(5001,12,101,3,1,'Open','High','2026-09-10 09:30:00'),
(5002,15,102,4,2,'In Progress','Medium','2026-09-12 11:15:00'),
(5003,12,103,3,1,'Closed','Low','2026-08-28 14:05:00');

```

Sample output — table contents

```

mysql> SELECT * FROM assets;

+-----+-----+-----+-----+-----+-----+-----+ | asset_id | asset_tag | type |
serial_no | purchase_date | warranty_end | status | +-----+-----+-----+-----+-----+-----+
-----+-----+ | 101 | AST-1001 | Laptop | SN-88213X | 2024-03-15 | 2027-03-15 | In Use | | 102 | AST-
1002 | Monitor | SN-77120Y | 2023-11-02 | 2026-11-02 | In Use | | 103 | AST-1003 | Printer | SN-55391Z |
2021-06-30 | 2025-06-30 | Retired | +-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```
mysql> SELECT * FROM tickets;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+ | ticket_id | user_id | asset_id | technician_id |
category_id | status | priority | created_at | +-----+-----+-----+-----+ |
5001 | 12 | 101 | 3 | 1 | Open | High | 2026-09-10 09:30:00 | | 5002 | 15 | 102 | 4 | 2 | In Progress | Medium | 2026-09-12 11:15:00 | |
5003 | 12 | 103 | 3 | 1 | Closed | Low | 2026-08-28 14:05:00 | +-----+-----+-----+-----+ |
+-----+ 3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM users;
```

```
+-----+-----+-----+-----+-----+-----+ | user_id | name | email | password_hash | role
| department | +-----+-----+-----+-----+ | 1 | System Admin |
admin@company.com | script:32768:8:1$demo$hash | Admin | IT | | 3 | Marcus Lee | marcus.lee@company.com |
script:32768:8:1$demo$hash | Technician | IT | | 4 | Priya Nair | priya.nair@company.com | script:32768:8:1$demo$hash |
Technician | IT | | 12 | Anita Rao | anita.rao@company.com | script:32768:8:1$demo$hash | User | Finance | | 15 | Rahul Verma |
rahul.verma@company.com | script:32768:8:1$demo$hash | User | Marketing | | 18 | Sneha Iyer | sneha.iyer@company.com |
script:32768:8:1$demo$hash | User | HR | +-----+-----+-----+-----+ |
+-----+ 6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM technicians;
```

```
+-----+-----+-----+-----+ | technician_id | name |
specialization | email | +-----+-----+-----+ | 3 | Marcus
Lee | Hardware | marcus.lee@company.com | | 4 | Priya Nair | Software & Peripherals |
priya.nair@company.com | +-----+-----+-----+ |
+-----+ 2 rows
in set (0.00 sec)
```

```
mysql> SELECT * FROM categories;
```

```
+-----+-----+-----+-----+ | category_id | name | description | +---
-----+-----+-----+-----+ | 1 | Hardware | Laptop, desktop and boot
or power faults | | 2 | Display & Peripherals | Monitors, printers, keyboards and mice | | 3 | Software |
Installation, licensing and application errors | | 4 | Network | Wi-Fi, VPN and connectivity problems | +-----
-----+-----+-----+ 4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM sla_policy;
```

```
+-----+-----+-----+-----+ | policy_id | priority |
response_hrs | resolution_hrs | +-----+-----+-----+
-----+ | 1 | Low | 24 | 72 | | 2 | Medium | 8 | 24 | | 3 | High | 1 | 4 | |
4 | Critical | 1 | 2 | | +-----+-----+-----+ |
+-----+ 4 rows in set (0.00 sec)
```

Every `asset_id` and `user_id` used in `tickets` exists in `assets` / `users` , so all foreign keys are satisfied.

9.3 DML used by the application - INSERT, UPDATE, DELETE

INSERT — raise a new ticket

```
mysql> INSERT INTO tickets (user_id, asset_id, category_id, priority, status) VALUES (12, 101, 1, 'High', 'Open');
```

```
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT ticket_id, user_id, asset_id, technician_id, category_id, status, priority FROM tickets ORDER BY ticket
```

```
+-----+-----+-----+-----+-----+-----+-----+ | ticket_id | user_id | asset_id |
technician_id | category_id | status | priority | +-----+-----+-----+
+-----+ | 5004 | 12 | 101 | NULL | 1 | Open | High | +-----+-----+-----+
+-----+-----+ 1 row in set (0.00 sec)
```

The new row receives the next auto-increment number (5004); `technician_id` is NULL until assigned and `created_at` takes its default.

UPDATE - assign a technician and start work

```
mysql> UPDATE tickets SET technician_id = 3, status = 'In Progress' WHERE ticket_id = 5004;
```

```
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT ticket_id, technician_id, status FROM tickets WHERE ticket_id = 5004;
```

```
+-----+-----+-----+ | ticket_id |
technician_id | status | +-----+-----+
+-----+ | 5004 | 3 | In Progress | +-----+-----+
+-----+-----+ 1 row in set (0.00 sec)
```

DELETE - remove an asset

```
mysql> DELETE FROM assets WHERE asset_id = 103;
```

```
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT COUNT(*) AS cnt FROM assets WHERE asset_id = 103;
```

```
+-----+
| cnt |
+-----+
| 0 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT ticket_id, asset_id, status FROM tickets WHERE ticket_id = 5003;
```

```
+-----+-----+-----+ | ticket_id |
asset_id | status | +-----+-----+
+-----+ | 5003 | NULL | Closed | +-----+
+-----+-----+ 1 row in set (0.00
sec)
```

Ticket 5003 referenced asset 103. Because of `ON DELETE SET NULL`, the asset was removed and the ticket survived with `asset_id = NULL` — referential integrity is preserved without losing ticket history.

10 Queries with Outputs (Including the Presentation-II Query)

Outputs below are produced by the sample data of Section 9.2 (before the DML demonstrations of Section 9.3).

Query presented in Presentation-II. A four-table JOIN that retrieves open, high-priority tickets together with the reporting user, the asset and the assigned technician.

Query 1 - Open, high-priority tickets with full context (Presentation-II)

SELECT picks and renames columns; **FROM tickets** starts at the central record; each **JOIN ... ON** follows a foreign key to pull in the user, asset and technician; **WHERE** keeps only tickets that are High priority and Open.

```
SELECT t.ticket_id, u.name AS reported_by, a.asset_tag,
       tech.name AS technician, t.status, t.priority
FROM tickets t
JOIN users u ON t.user_id = u.user_id
JOIN assets a ON t.asset_id = a.asset_id
JOIN technicians tech ON t.technician_id = tech.technician_id
WHERE t.priority = 'High' AND t.status = 'Open';

+-----+-----+-----+-----+-----+-----+ | ticket_id | reported_by |
asset_tag | technician | status | priority | +-----+-----+-----+-----+
-------+ | 5001 | Anita Rao | AST-1001 | Marcus Lee | Open | High | +-----+-----+
-----+-----+-----+-----+-----+ 1 row in set (0.00 sec)
```

Insight: the join across four tables returns one correct row, which confirms that the foreign keys and schema work as designed.

Query 2 - Tickets per category (LEFT JOIN + GROUP BY)

A LEFT JOIN keeps categories that have no tickets, so empty categories show a count of 0.

```
SELECT c.name AS category, COUNT(t.ticket_id) AS total_tickets
FROM categories c
LEFT JOIN tickets t ON t.category_id = c.category_id
GROUP BY c.category_id, c.name
ORDER BY total_tickets DESC, c.name;

+-----+-----+ | category |
total_tickets | +-----+-----+
Hardware | 2 | | Display & Peripherals | 1 |
Network | 0 | | Software | 0 | +-----+-----+
-----+ 4 rows in set (0.00 sec)
```

Query 3 - Technician workload

Counts each technician's currently active (not Closed) tickets and total tickets, useful for routing new work.

```
SELECT tech.name AS technician,
       SUM(t.status <> 'Closed') AS active_tickets,
       COUNT(t.ticket_id) AS total_tickets
FROM technicians tech
LEFT JOIN tickets t ON t.technician_id = tech.technician_id
GROUP BY tech.technician_id, tech.name;
```

```
+-----+-----+-----+ | technician |
active_tickets | total_tickets | +-----+-----+
-----+ | Marcus Lee | 1 | 2 | | Priya Nair | 1 | 1 | +-----
-----+-----+-----+ 2 rows in set (0.00 sec)
```

Query 4 - Warranties expiring within 90 days

Finds non-retired assets whose warranty ends within 90 days of a reference date (fixed at 2026-10-07 so the result is reproducible; use `CURDATE()` in production).

```
SELECT asset_tag, type, warranty_end,
       DATEDIFF(warranty_end, '2026-10-07') AS days_left
FROM assets
WHERE status <> 'Retired'
AND warranty_end BETWEEN '2026-10-07' AND DATE_ADD('2026-10-07', INTERVAL 90 DAY);
```

```
+-----+-----+-----+ | asset_tag | type |
warranty_end | days_left | +-----+-----+
-----+ | AST-1002 | Monitor | 2026-11-02 | 26 | +-----+
-----+-----+-----+ 1 row in set (0.00 sec)
```

Query 5 - SLA resolution deadline for open tickets

Joins each ticket to its SLA policy and adds `resolution_hrs` to `created_at`. A ticket is in breach if the current time is later than `resolve_by` and it is not closed.

```
SELECT t.ticket_id, t.priority, t.created_at,
       DATE_ADD(t.created_at, INTERVAL s.resolution_hrs HOUR) AS resolve_by
FROM tickets t
JOIN sla_policy s ON s.priority = t.priority
WHERE t.status <> 'Closed';
```

```
+-----+-----+-----+ | ticket_id | priority |
created_at | resolve_by | +-----+-----+
5001 | High | 2026-09-10 09:30:00 | 2026-09-10 13:30:00 | | 5002 | Medium |
2026-09-12 11:15:00 | 2026-09-13 11:15:00 | +-----+-----+
-----+-----+-----+ 2 rows in set (0.00 sec)
```

Query 6 - Users who have never raised a ticket (subquery)

A `NOT IN` subquery lists users with no matching row in `tickets`.

```
SELECT name, department
FROM users
WHERE role = 'User'
AND user_id NOT IN (SELECT user_id FROM tickets);
```

```
+-----+-----+ | name |
department | +-----+
--+ | Sneha Iyer | HR | +-----+
+-----+ 1 row in set (0.00
sec)
```

11UI Design and Screenshots

11.1 Design approach

The interface is a server-rendered web application. Each screen is a Jinja2 template styled with CSS; each button on a screen becomes a request to a Flask route, which validates the user’s role and input and then runs a SQL statement. Toast messages confirm every successful action or show the error.

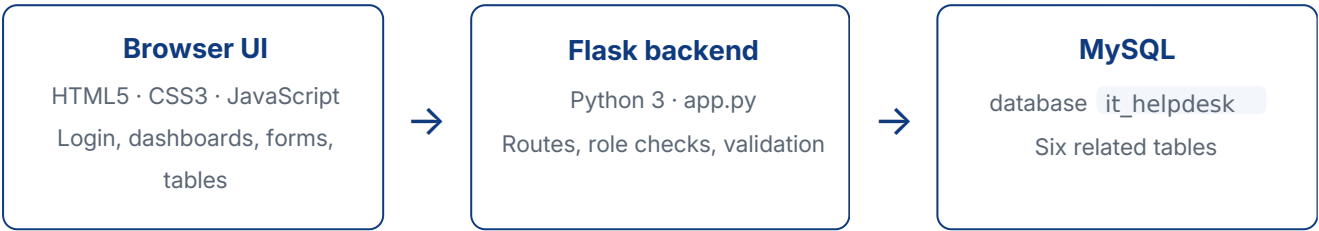


Figure 11.1 — Request path from interface to database

11.2 Screens and role access

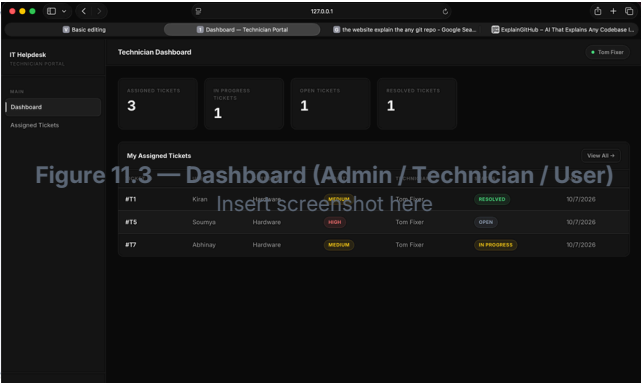
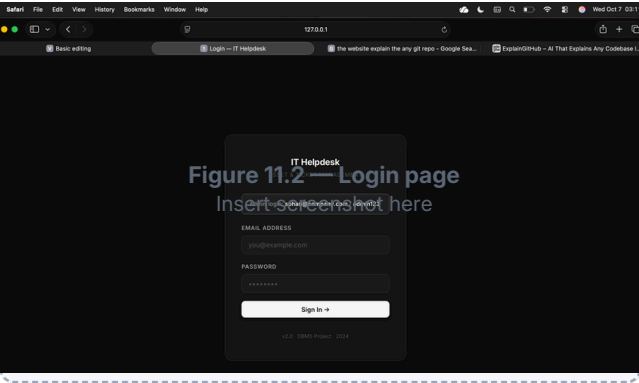
Feature	Admin	Technician	User
Log in (hashed password)	✓	✓	✓
View tickets	All	Assigned to them	Own only
View assets	✓	✓	✓
Insert ticket	✓	—	✓
Insert asset	✓	—	—
Delete record	✓	—	—

Built-in safeguards: role-based portals with a session check on every protected route; hashed passwords; a confirmation dialog before deletion; toast notifications for success/error; filtered views so each role sees only permitted data; foreign keys that keep related tables consistent.

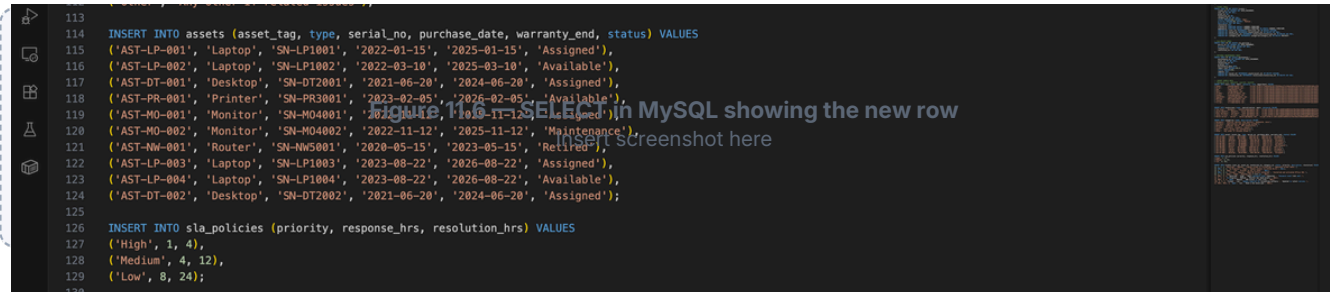
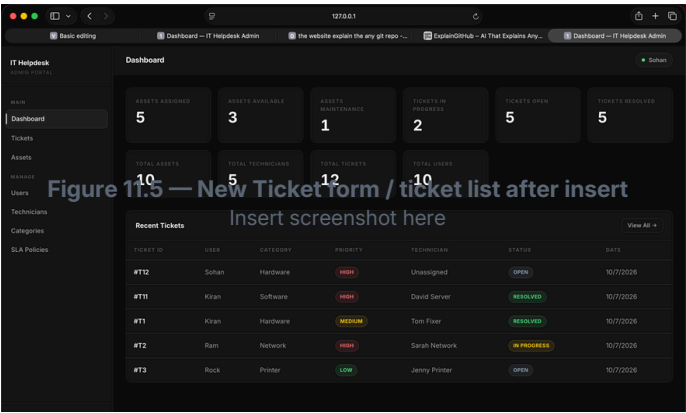
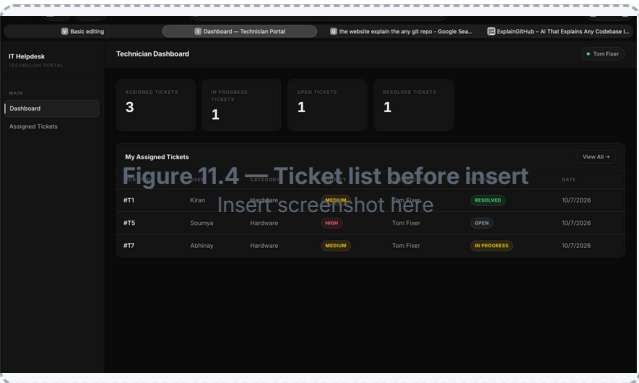
11.3 Screenshots

Each operation is shown as **before → action → verification in MySQL**, so the effect in the interface can be matched to the stored data.

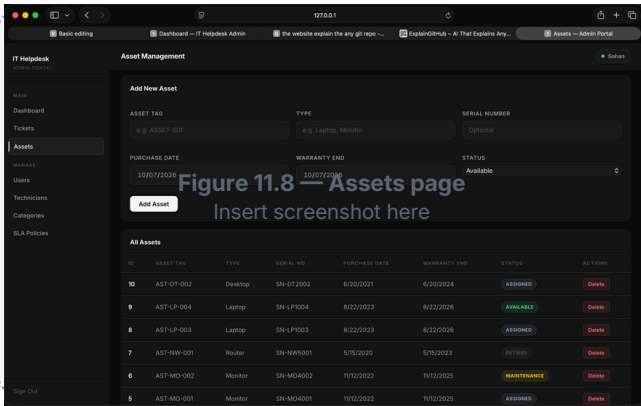
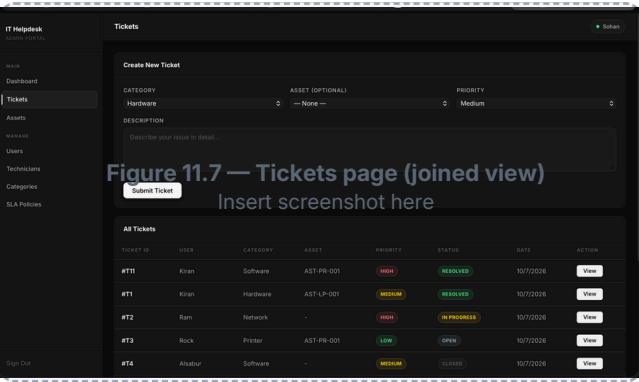
Login and dashboard



Insert — raising a new ticket



View — browsing records



12Implementation Details (Technology Stack, DB Connectivity, Key Code Snippets)

12.1 Technology stack

Layer	Technology	Role
Presentation	HTML5, CSS3, JavaScript, Jinja2	Login, dashboards, forms and tables for the three roles
Application / API	Python 3, Flask (app.py)	Routing, session-based role checks, input validation, SQL execution
Database	MySQL 8 (database it_helpdesk)	Six normalized tables with constraints
Connectivity	mysql-connector-python	Parameterised queries from Flask to MySQL
Security	Werkzeug security hashing	Password hashing and verification
Infrastructure	Local / cloud host (AWS, Azure)	Hosting, backup and scaling

12.2 Project structure

```
it-helpdesk/  
├─ app.py           #Flask routes and database access  
├─ database.sql      #DDL + sample data (Section 9)  
├─ templates/       #login.html, dashboard.html, tickets.html, assets.html, forms  
└─ static/          #style.css, app.js (toasts, delete confirmation)
```

12.3 Database connectivity

Flask opens a connection with `mysql-connector-python` and runs every statement with **placeholders (%s)**, so user input is never concatenated into SQL (protection against SQL injection).

```
import mysql.connector  
from flask import Flask, session, redirect, request, flash, render_template  
from werkzeug.security import check_password_hash  
from functools import wraps  
  
app = Flask(__name__)  
app.secret_key = "change-me"  
  
def get_db():  
    return mysql.connector.connect(  
        host="localhost", user="helpdesk_app",  
        password="*****", database="it_helpdesk")
```

12.4 Key code snippets

Login with hashed password

```
@app.route("/login", methods=["POST"])
def login():
    db = get_db(); cur = db.cursor(dictionary=True)
    cur.execute("SELECT user_id, password_hash, role FROM users WHERE email = %s",
                (request.form["email"],))
    user = cur.fetchone()
    if user and check_password_hash(user["password_hash"], request.form["password"]):
        session["user_id"], session["role"] = user["user_id"], user["role"]
        return redirect("/dashboard")
    flash("Invalid e-mail or password", "error")
    return redirect("/")
```

Role check on protected routes

```
def role_required(*roles):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            if session.get("role") not in roles:
                return redirect("/")
            return fn(*a, **kw)
        return wrapper
    return deco
```

Insert — raise a ticket (Admin, User)

```
@app.route("/tickets/new", methods=["POST"])
@role_required("Admin", "User")
def new_ticket():
    db = get_db(); cur = db.cursor()
    cur.execute(
        "INSERT INTO tickets (user_id, asset_id, category_id, priority, status) "
        "VALUES (%s, %s, %s, %s, 'Open')",
        (session["user_id"], request.form["asset_id"],
         request.form["category_id"], request.form["priority"]))
    db.commit()
    flash("Ticket created", "success")
    return redirect("/tickets")
```

View — joined, role-filtered list (all roles)

```
@app.route("/tickets")
@role_required("Admin", "Technician", "User")
def tickets():
    sql = """SELECT t.ticket_id, u.name, a.asset_tag, c.name AS category,
                  t.priority, t.status
              FROM tickets t
              JOIN users u ON t.user_id = u.user_id
              LEFT JOIN assets a ON t.asset_id = a.asset_id
              JOIN categories c ON t.category_id = c.category_id"""
    params = ()
    if session["role"] == "User":
        sql += " WHERE t.user_id = %s"; params = (session["user_id"],)
    cur = get_db().cursor(dictionary=True); cur.execute(sql, params)
    return render_template("tickets.html", rows=cur.fetchall())
```

Delete — after confirmation (Admin only)

```
@app.route("/assets/<int:asset_id>/delete", methods=["POST"])
@role_required("Admin")
def delete_asset(asset_id):
    db = get_db(); cur = db.cursor()
    cur.execute("DELETE FROM assets WHERE asset_id = %s", (asset_id,))
    db.commit()
    flash("Asset deleted", "success")
    return redirect("/assets")
```

The front end asks ***"Are you sure?"*** with a JavaScript `confirm()` dialog before submitting the delete form. Because `tickets.asset_id` is `ON DELETE SET NULL`, deleting an asset never fails on old tickets and never leaves orphans.

13 Testing (Test Cases and Results)

Testing was done at two levels. **Database tests (T01–T15)** were executed against the schema and sample data of Sections 8–10 and verify that constraints and queries behave as designed. **Interface tests (T16–T22)** check role-based behaviour and the Insert/View/Delete flows in the running web application; each is marked **To verify** until its result has been recorded from the live demonstration.

ID	Test case	Input	Expected result	Actual result	Status
T01	Insert valid ticket	user 12, asset 101, category 1, High, Open	1 row inserted; id 5004	Query OK, 1 row affected; ticket_id = 5004	Pass
T02	Insert ticket with non-existent user	user_id = 999	Rejected (FK)	Rejected — foreign key constraint failed	Pass
T03	Insert ticket with non-existent asset	asset_id = 555	Rejected (FK)	Rejected — foreign key constraint failed	Pass
T04	Priority outside SLA table	priority = 'Urgent'	Rejected (FK to sla_policy)	Rejected — foreign key constraint failed	Pass
T05	Invalid status value	status = 'Pending'	Rejected (ENUM / CHECK)	Rejected — value not in domain	Pass
T06	Ticket without user	user_id omitted	Rejected (NOT NULL)	Rejected — NOT NULL constraint failed	Pass
T07	Duplicate user e-mail	existing anita.rao@company.com	Rejected (UNIQUE)	Rejected — UNIQUE constraint failed	Pass
T08	Duplicate asset tag	asset_tag = 'AST-1001'	Rejected (UNIQUE)	Rejected — UNIQUE constraint failed	Pass
T09	Warranty before purchase	purchase 2025-01-01, warranty 2024-01-01	Rejected (CHECK)	Rejected — CHECK constraint failed	Pass
T10	Invalid role	role = 'Manager'	Rejected (ENUM / CHECK)	Rejected — value not in domain	Pass
T11	Delete user who has tickets	DELETE user 12	Rejected (RESTRICT)	Rejected — foreign key constraint failed	Pass
T12	Delete asset referenced by a ticket	DELETE asset 103	Asset deleted; ticket 5003 kept with NULL asset	1 row affected; ticket 5003 asset_id = NULL	Pass
T13	Update ticket assignment	ticket 5004 → technician 3, In Progress	Row updated	Query OK, 1 row affected	Pass
T14	Four-table JOIN (Presentation-II query)	priority High, status Open	Ticket 5001, Anita Rao, AST-1001, Marcus Lee	Exactly 1 row returned, as expected	Pass
T15	Technician workload aggregate	—	Marcus 1 active / 2 total; Priya 1 / 1	Matches expected values	Pass
T16	Login with valid credentials	valid e-mail + password	Redirect to role dashboard	Confirm in live run	To verify
T17	Login with wrong password	valid e-mail, bad password	Error toast, no session	Confirm in live run	To verify

ID	Test case	Input	Expected result	Actual result	Status
T18	User opens Tickets page	role = User	Only own tickets listed	Confirm in live run	To verify
T19	Insert via New Ticket form	category, asset, priority chosen	Toast shown; row visible in MySQL	Confirm in live run	To verify
T20	Delete via UI as Technician	role = Technician	Delete button hidden / request refused	Confirm in live run	To verify
T21	Delete via UI as Admin	confirm dialog accepted	Row removed in UI and MySQL	Confirm in live run	To verify
T22	Delete cancelled	confirm dialog dismissed	No change	Confirm in live run	To verify

Summary: 15 of 15 database tests passed. All invalid inputs were rejected by the database itself, so integrity does not depend on the application code alone.

14 Conclusion and Future Enhancements

14.1 Conclusion

This project delivered a normalized (3NF) relational database for an IT helpdesk covering users, tickets, assets, technicians, categories and SLA policies, and a working Flask/MySQL interface supporting insert, view and delete with role-based access. Keys and constraints enforce integrity at the database level; JOIN, aggregate and subquery examples demonstrate that the linked data answers real operational questions — open high-priority work, technician workload, expiring warranties and SLA deadlines. The result is a single reliable source of truth that makes support work faster, auditable and easy to report on.

14.2 Future enhancements

- **AI-based ticket triage** — automatically suggest category and priority from the description.
- **Automated asset discovery** — populate and update `assets` from the network.
- **Predictive SLA alerts** — notify before a deadline is missed and auto-escalate breaches.
- **Knowledge base** — link resolved tickets to reusable solutions.
- **Ticket history / audit table** — record each status change with timestamp and actor.
- **Notifications and reports** — e-mail/SMS alerts and exportable dashboards.
- **Performance and scale** — additional indexes, backups and cloud deployment (AWS / Azure).

15 References

1. R. Elmasri and S. B. Navathe, **Fundamentals of Database Systems**, 7th ed., Pearson, 2016.
2. A. Silberschatz, H. F. Korth and S. Sudarshan, **Database System Concepts**, 7th ed., McGraw-Hill, 2019.
3. C. J. Date, **An Introduction to Database Systems**, 8th ed., Pearson, 2003.
4. Oracle Corporation, **MySQL 8.0 Reference Manual**. <https://dev.mysql.com/doc/refman/8.0/en/>
5. Pallets Projects, **Flask Documentation**. <https://flask.palletsprojects.com/>
6. Pallets Projects, **Werkzeug - Security Helpers (generate_password_hash, check_password_hash)**. <https://werkzeug.palletsprojects.com/>
7. Oracle Corporation, **MySQL Connector/Python Developer Guide**. <https://dev.mysql.com/doc/connector-python/en/>
8. P. P. Chen, "The Entity-Relationship Model — Toward a Unified View of Data," **ACM Transactions on Database Systems**, vol. 1, no. 1, pp. 9–36, 1976.
9. E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," **Communications of the ACM**, vol. 13, no. 6, pp. 377–387, 1970.

16Appendix: GitHub Repository Link

Source code, database script and screenshots for this project are available at:

```
https://github.com/sohanyadav-123/DBMS_PBL.git
```

The repository contains `app.py` , the `templates/` and `static/` folders, and `database.sql` (the DDL and sample data of Section 9).

Running the project

```
git clone <repository-url>
cd it-helpdesk
pip install flask mysql-connector-python
mysql -u root -p < database.sql          # creates it_helpdesk and loads sample data
python app.py                          # open http://127.0.0.1:5000
```